Funciones Friend

Esta herramienta que nos provee C++ viene a romper el concepto de encapsulamiento. Por un lado, estamos encapsulando para proteger los datos a asignar en una variable y una función friend salta el encapsulamiento y nos permite acceder a miembros privados directamente. Una función friend es una función que, aunque no sea un miembro de una clase, tiene acceso a sus miembros privados y protegidos. Para declarar una función como amiga de una clase, se utiliza la palabra clave ***friend*** en la definición de la clase.

Características clave

- No es miembro de la clase: Una función friend no es un miembro de la clase; por lo tanto, no se invoca utilizando un objeto de la clase.
- Acceso total: Las funciones friend pueden acceder a todos los miembros privados y protegidos de la clase.
- No transitoria: La amistad entre una función y una clase no es heredada. Es decir, las clases derivadas no heredan las funciones friend de sus clases base.
- Declaración explícita: La función friend debe ser declarada explícitamente dentro de la clase que concede la amistad.

Casos de aplicación

- Operadores sobrecargados como funciones friend. En muchos casos, se utilizan funciones friend para sobrecargar operadores que requieren acceso a los atributos privados de dos o más objetos de una clase. Por ejemplo, si queremos sobrecargar el operador de suma (+) para sumar dos objetos de una clase, la función amiga puede acceder directamente a los atributos privados.
- Interacción entre clases. En ocasiones, es necesario que dos o más clases interactúen de manera que accedan mutuamente a sus datos privados. Las funciones friend permiten que las clases colaboren sin violar la encapsulación.
- Depuración y acceso controlado. En situaciones donde se requiere acceder a datos privados durante la depuración o pruebas, se pueden usar funciones friend para otorgar acceso temporal a los datos, sin exponerlos directamente. Esto permite controlar qué funciones específicas tienen acceso a esos datos y en qué contexto.

Veamos algo practico y sencillo, tenemos una clase persona con un atributo edad y necesitamos comparar las edades entre dos objetos persona. Hasta el momento estaríamos haciendo algo así:

```
class persona{
private:
    int edad;
public:
    persona(int e){edad=e;}
    int get_edad(){return edad;}
};

int main(){
    persona p1(12);
    persona p2(65);

    if(p1.get_edad()>p2.get_edad())
        cout<<"Persona 1 es mayor"<<endl;
    else
        cout<<"Persona 2 es mayor"<<endl;
}
```

Tenemos que acceder a cada atributo y realizar una comparación, o bien podríamos crear un método dentro de la clase que compare las edades:

```
class persona{
private:
    int edad;
public:
    persona(int e){edad=e;}
    int get_edad(){return edad;}
    void comparar(persona);
};

void persona::comparar(persona p2)
{
    if(this->edad>p2.edad)
        cout<<"Persona 1 es mayor"<<endl;
    else
        cout<<"Persona 2 es mayor"<<endl;
}

int main(){
    persona p1(12);
    persona p2(65);
    p1.comparar(p2);
}
```

Pero tampoco queda practico, ya que para realizar la comparación estamos accediendo desde uno de los objetos para invocar al método. Mas prolijo quedaría tener una función externa que compare estas edades sin necesidad de ingresar a los objetos. Algo como lo siguiente:

```
class persona{
private:
    int edad;
public:
    persona(int e){edad=e;}
    int get_edad(){return edad;}
};

void comparar(persona p1,persona p2)
{
    if(p1.edad>p2.edad)
        cout<<"Persona 1 es mayor"<<endl;
    else
        cout<<"Persona 2 es mayor"<<endl;
}

int main(){
    persona p1(12);
    persona p2(65);
    comparar(p1,p2);
}
```

Ejecutando este código obtenemos un error ya que el atributo edad es privado y no podemos acceder desde una función externa.

```
C:\Users\u604...  16    error: 'int persona::edad' is private within this context
```

Ahora sí, convertimos la función recién creada en una función amiga de la clase la cual puede acceder a los atributos privados.

```
class persona{
private:
    int edad;
public:
    persona(int e){edad=e;}
    int get_edad(){return edad;}
    friend void comparar(persona ,persona );
};
```

Además de funciones externas como el caso que vimos recién, también se puede hacer amiga de la clase métodos de otra clase o directamente una clase entera. Vemos el siguiente ejemplo, por un lado, en la clase persona hacemos amiga por completo la otra clase, y por otro lado en la otra clase hacemos amiga una función propia de la clase persona.

```
class otra_Clase;

class persona{
private:
    int edad;
public:
    persona(int e){edad=e;}
    int get_edad(){return edad;}
    friend otra_Clase;
};

class otra_Clase
{
public:
    otra_clase(){}
    friend int persona::get_edad();
};
```